

Checkout.com Mobile Challenge - iOS

Build a mobile app that does 3D Secure payments

Guidance

Payment processing is complicated. If you encounter any terminology you are unfamiliar with, please do research online using your search engine of choice. Please do read this entire document before starting this exercise. **It should take you around 4 hours to complete.**

Test card details here: <https://docs.checkout.com/docs/testing#section-credit-cards>

Background

Fraud and Chargebacks are not great for merchants in an E-Commerce environment. They need a way to protect their business and avoid liability. 3D Secure is a great solution because it adds an additional layer of security to online payments.

We would like you to build a mobile app that will support a full payment interaction with 3D Secure.

Processing a payment with 3D Secure involves multiple elements:



1. **Client:** The mobile app, where the visual interaction will occur
2. **API:** A server-side API that will perform a transaction and return back a 3D Secure URL

Requirements

Prerequisites

Use these keys below to make the following network requests in the sandbox environment.

- Public key for Part 1: **pk_sbox_gnrjo6pl5azfmgdnrfrbbejo7ev**
- Private key for Part 2: **sk_sbox_bvzfhwhivsgj33smfjjeb6t64i4**

How to set up the authentication for network requests can be found in the Security section of each request's details in the links.

Part 1: An app screen that accepts card details input

It should include the following elements:

- an input field for card number
- an input field for expiration date
- an input field for cvv
- a pay button

Upon completing the card details, clicking the pay button should

- trigger a HTTP POST request to our sample back end API:
<https://api-reference.checkout.com/#operation/requestAToken>
- With the example JSON body below

JSON

```
{
  "type": "card"
  "number": "4242424242424242"
  "expiry_month": "10",
  "expiry_year": "2025",
  "cvv": "100",
}
```

- Keep the **token** field from the JSON response to make a payment in Part 2, example JSON body listed below

JSON

```
{
  "token": "tok_ubfj2q76miwundwlk72vxt2i7q",
  // other fields won't be used
}
```

Part 2: Making a Payment Request after tokenisation is completed

- Use `token` received from part 1 to make a payment to this end point:
<https://api-reference.checkout.com/#operation/requestAPaymentOrPayout>
- The request will follow the specs below
 - `source.type` will be `token`
 - `Currency` will be `GBP`
 - `3ds.enabled` will be `true`
 - `success_url` will be set to a hardcoded url by your choice, this will be used as a redirection URL after 3DS challenge succeed
 - `failure_url` will be set to a hardcoded url by your choice, this will be used as a redirection URL after 3DS challenge failed
- the example request body should look like something below

JSON

```
{
  "source": {
    "type": "token",
    "token": "tok_4gzeau5o2uqubbk6fufs3m7p54",
  },
  "amount": 6540,
  "currency": "GBP",
  "3ds": {
    "enabled": true,
  },
  "success_url": "https://example.com/payments/success",
  "failure_url": "https://example.com/payments/fail",
}
```

Part 3: An app screen that handles a 3DS URL and shows the payment outcome

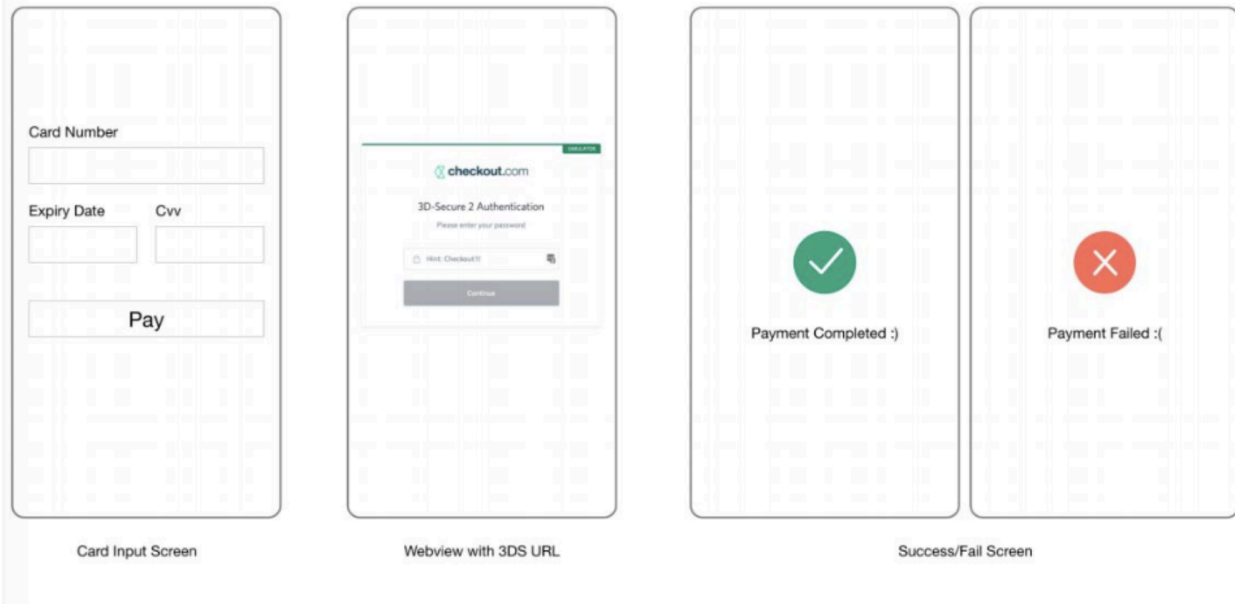
- The payment response from part 2 should contain a redirection URL to the 3DS challenge and the `status` field should be `Pending`
- Use `_links.redirect.href` field from the response to trigger a Web View of the 3DS verification page. (Sample example listed below)
- After the 3DS verification is done, the web page will automatically redirect to the success or the fail page sent in the API call done in Part 1 .
- Detect this success or failure and then dismiss the WebView, and then show a native success or fail screen.
- Tips
 - To test a successful interaction, you can use these card details:
 - Number: 4242-4242-4242-4242
 - Date: 06/2030
 - Cvv: 100
 - To test an unsuccessful interaction, you can use these card details:
 - Number:
 - 4243-7542-7170-0719
 - Date: 06/2030
 - Cvv: 100

JSON

```
{
  "status": "Pending",
  "_links": {
    "redirect": {
      "href":
"https://api.checkout.com/3ds/pay_mbabizu24mvu3mela5njyhpit4",
    }
  }
}
```

Sample UI Mock

Free to design this screen in any way you want.



Considerations

- Include whatever documentation/notes you feel is appropriate, this should include some details of assumptions made, areas you feel could be improved, tests etc.
- Different card schemes (Visa, Amex) have different ways of formatting the cards and different validation requirements.
- Potentially consideration:
 - Formatting the card input field
 - Showing the scheme icon when a certain type of card is detected
 - Doing validation based on the card type

Deliverables

A runnable mobile app project that will allow the full payment experience using 3D Secure

Assessment Aspects

Must Have

- User Experience
- Code maintainability, readability and scalability
- Testability
- Error Handling

Nice to Have

- Accessibility
- Localisation

Tech Stack

Must Have

- SwiftUI
- MVVM
- Swift concurrency - Async await
- Unit test - XCTest
- SPM
- Application and SPM target iOS 15 (supporting iOS only)
- Code Documentation
- ReadMe

Nice to Have

- Any suitable design pattern
- Code Linting
- Git history

Avoid the following

- MacOS - WatchOS - VisionOS - UITest - Combine - Objective C - UIKit - Cocoapods - Any external dependencies